

TP02 Report — Dependency Injection & MVC Pattern in NestJS

▣ Project Overview

This project is a **NestJS backend** that implements a **Receipt Management API** with full CRUD operations. It demonstrates core NestJS concepts: **Dependency Injection (DI)**, **MVC pattern**, **TypeORM with PostgreSQL**, **Validation**, **Guards**, and **Interceptors**.

Tech Stack

Technology	Purpose
NestJS (v11)	Backend framework (built on top of Express.js). It uses TypeScript and follows Angular-style architecture with modules, controllers, and services.
TypeORM (v0.3)	Object-Relational Mapping — lets you interact with the database using TypeScript classes instead of writing raw SQL queries.
PostgreSQL 16	Relational database running inside a Podman container. Stores the receipt table.
class-validator	Library that provides validation decorators (@IsString() , @IsNumber() , etc.) to validate incoming request data.
class-transformer	Works with class-validator — it transforms plain JSON objects from the request body into actual class instances so that decorators can be applied.
pg	The PostgreSQL client driver for Node.js. TypeORM uses this internally to communicate with PostgreSQL.
rxjs	Reactive Extensions for JavaScript — provides Observable pattern. Used by NestJS interceptors to handle async streams of data.
dotenv	Loads environment variables from a .env file into process.env so we can configure the app without hardcoding values.

□ Project Structure

```
src/
├── main.ts                # App entry point – bootstraps the NestJS
application
├── app.module.ts          # Root module – connects everything
together
├── app.controller.ts      # Root controller – handles GET / (Hello
World)
├── app.service.ts         # Root service – provides getHello()
method
├── common/
│   ├── guards/
│   │   └── api-key.guard.ts    # API Key authentication guard –
protects routes
│   ├── interceptors/
│   │   └── logging.interceptor.ts # HTTP request logging – logs method,
URL, duration
│   └── database/
│       ├── entities/
│       │   └── receipts.entity.ts # Receipt entity – maps to PostgreSQL
'receipt' table
│   └── receipts/
│       ├── receipts.module.ts    # Receipts feature module – bundles
receipt-related code
│       ├── receipts.controller.ts # Receipts CRUD controller – defines HTTP
endpoints
│       └── receipts.service.ts    # Receipts service – contains business
logic
│   └── dto/
│       ├── create-receipt.dto.ts # DTO for creating a receipt (all fields
required)
│       └── update-receipt.dto.ts  # DTO for updating a receipt (all fields
optional)
```

Why this structure?

NestJS follows a **modular architecture**. Each feature (like receipts) has its own module, controller, service, and DTOs. This keeps code **organized**, **reusable**, and **easy to maintain**. The `common/` folder stores shared utilities (guards, interceptors) that can be reused across multiple modules.

□ Key Concepts Explained In-Depth

1. Dependency Injection (DI)

What is it? Dependency Injection is a design pattern where a class **does NOT create its own dependencies**. Instead, the dependencies are **provided (injected) from the outside** by a framework or container. In NestJS, this is handled by the **IoC (Inversion of Control) container**.

Without DI (Bad Practice):

```
// ❌ BAD – Controller creates its own service
class ReceiptsController {
  private service = new ReceiptsService(); // tightly coupled!
}
```

Problems: - Controller is **tightly coupled** to ReceiptsService - Cannot easily swap ReceiptsService with a mock for testing - If ReceiptsService constructor changes, every file that creates it must change

With DI (What we use):

```
// ✅ GOOD – NestJS injects the service automatically
@Controller('receipts')
export class ReceiptsController {
  constructor(private readonly receiptsService: ReceiptsService) {}
  //      NestJS sees the type ReceiptsService in the constructor
  //      and automatically provides an instance
}
```

Benefits: - **Loose coupling** — Controller doesn't know how Service is created - **Testability** — Easy to replace with a mock service in unit tests - **Single Responsibility** — Each class focuses on one job - **Lifecycle management** — NestJS manages when to create/destroy instances

How does NestJS know what to inject? 1. You mark a class with **@Injectable()** — this tells NestJS "this class can be provided as a dependency" 2. You register it as a **provider** in a module — **providers: [ReceiptsService]** 3. NestJS reads the constructor parameter types and matches them with registered providers 4. NestJS automatically creates an instance and passes it to the constructor

Where is DI used in this project?

Class	Injects	How
AppController	AppService	constructor(private readonly appService: AppService)
ReceiptsController	ReceiptsService	constructor(private readonly receiptsService: ReceiptsService)

Class	Injects	How
ReceiptsService	Repository<Receipt>	@InjectRepository(Receipt) private readonly receiptRepo: Repository<Receipt>

What is @InjectRepository(Receipt)? TypeORM repositories are not regular NestJS services — they are generated dynamically. The `@InjectRepository(Receipt)` decorator tells NestJS to inject the **specific repository for the Receipt entity**. This repository provides methods like `find()`, `findOne()`, `save()`, `create()`, `remove()`.

2. MVC Pattern (Model-View-Controller)

What is MVC? MVC is an architectural pattern that separates an application into three concerns:

Layer	Responsibility	NestJS Equivalent	File in Project
Model	Data structure + database interaction	Entity + Repository	<code>receipts.entity.ts</code>
View	What the user sees (UI or response)	JSON response (no HTML in REST API)	Returned by controllers
Controller	Receives requests, delegates to service	Controller class	<code>receipts.controller.ts</code>
Service (bonus layer)	Business logic between Controller and Model	Service class	<code>receipts.service.ts</code>

Why separate Controller and Service? - **Controller** should ONLY handle HTTP concerns (reading params, body, setting status codes) - **Service** should contain the actual **business logic** (validation, calculations, database queries) - This means the same Service can be reused by multiple controllers, CLI commands, or background jobs

Detailed flow of POST /receipts:

1. Client sends: POST /receipts with JSON body { "name": "Coffee", "price": 4.50, "issuedAt": "2026-03-19T10:00:00Z" }
□
2. Guard (ApiKeyGuard): Checks x-api-key header → if invalid, returns 401 and STOPS
□
3. Interceptor (LoggingInterceptor): Records start time
□
4. Pipe (ValidationPipe): Validates body against CreateReceiptDto
 - Is "name" a non-empty string? □
 - Is "price" a number >= 0? □
 - Is "issuedAt" a valid date string? □
 - If ANY fails → returns 400 and STOPS
 □
5. Controller (create method): Receives validated DTO, calls

```

receiptsService.create(dto)
    []
6. Service (create method):
  - receiptRepo.create({...}) → creates Receipt object IN MEMORY
  - receiptRepo.save(receipt) → INSERT INTO receipt (...) VALUES (...) → writes to PostgreSQL
  - Returns the saved receipt (now has a generated UUID)
    []
7. Interceptor (after): Calculates elapsed time, logs "[HTTP] POST /receipts - 45ms"
    []
8. Response: 201 Created with the receipt JSON

```

3. What is a Module in NestJS?

A **Module** is a class decorated with `@Module()` that organizes the application into logical units. Every NestJS app has at least one module — the **root module** (`AppModule`).

```

@Module({
  imports: [...],      // Other modules this module depends on
  controllers: [...],  // Controllers that handle HTTP requests
  providers: [...],    // Services, repositories, and other injectables
  exports: [...],      // Providers to share with other modules
})

```

In this project: - `AppModule` — Root module, imports TypeORM config + `ReceiptsModule` - `ReceiptsModule` — Feature module, bundles everything related to receipts

Why modules? - **Encapsulation** — Each module contains its own controllers, services, entities - **Reusability** — A module can be imported into other modules - **Organization** — As the app grows, each feature gets its own module

4. What is an Entity?

An **Entity** is a TypeScript class that maps to a **database table**. Each instance of the entity represents a **row** in the table. TypeORM uses decorators to define columns, types, and constraints.

TypeScript Class	→	PostgreSQL Table
Receipt class	→	receipt table
receiptId: string	→	"receiptId" uuid PRIMARY KEY
issuedAt: Date	→	"issuedAt" timestamp NOT NULL
name: string	→	"name" varchar NOT NULL
price: number	→	"price" numeric(10,2) NOT NULL

5. What is a Repository?

A **Repository** is a TypeORM class that provides methods to interact with a specific entity's database table. You **don't write SQL** — you call repository methods instead.

Repository Method	SQL Equivalent	Description
<code>repo.find()</code>	<code>SELECT * FROM receipt</code>	Get all rows
<code>repo.find({ order: { issuedAt: 'DESC' } })</code>	<code>SELECT * FROM receipt ORDER BY "issuedAt" DESC</code>	Get all rows sorted
<code>repo.findOne({ where: { receiptId } })</code>	<code>SELECT * FROM receipt WHERE "receiptId" = \$1</code>	Get one row by condition
<code>repo.create({...})</code>	<i>(no SQL — creates object in memory)</i>	Create entity instance
<code>repo.save(entity)</code>	<code>INSERT INTO receipt (...)</code> <code>VALUES (...)</code> or <code>UPDATE receipt SET ...</code>	Insert new or update existing
<code>repo.remove(entity)</code>	<code>DELETE FROM receipt WHERE "receiptId" = \$1</code>	Delete a row

Important: `create()` vs `save()` - `create()` only makes a JavaScript object. **Nothing is written to the database yet.** - `save()` actually executes the SQL INSERT/UPDATE query. - You need BOTH: `create()` first, then `save()`.

□ File-by-File Detailed Explanation

`main.ts` — Application Entry Point

This is the first file that runs when you start the NestJS application.

```
import 'dotenv/config';
// □ Loads .env file into process.env BEFORE anything else runs.
// This is why it's the FIRST import – environment variables must be
// available when AppModule initializes (for DB connection, API_KEY, etc.)

import { NestFactory } from '@nestjs/core';
// □ NestFactory is a helper class that creates a NestJS application instance.

import { AppModule } from './app.module';
import { ValidationPipe } from '@nestjs/common';
import { LoggingInterceptor } from './common/interceptors/logging.interceptor';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
```

```

//   Creates the NestJS app by reading AppModule.
//   NestJS scans all imports, controllers, providers and builds the DI container.
//   This also establishes the database connection (TypeORM).

app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,
    //   STRIPS any properties from request body that are NOT in the DTO.
    //   Example: { "name": "Coffee", "hack": "evil" }
    //   → "hack" is not in CreateReceiptDto, so it gets removed
    //   → Only { "name": "Coffee" } passes through

    forbidNonWhitelisted: true,
    //   Instead of silently stripping unknown fields, it throws a 400 error.
    //   Example: { "name": "Coffee", "hack": "evil" }
    //   → Returns: 400 Bad Request "property hack should not exist"
    //   This is STRICTER than just whitelist.

    transform: true,
    //   Automatically converts the plain JSON object into a DTO class instance.
    //   Without this, @Body() gives you a plain object {}, not a CreateReceiptDto
instance.
    //   With this, class-validator decorators can properly validate the data.
    //   It also converts types: string "5" → number 5 if the DTO says @IsNumber().
  }),
);

app.useGlobalInterceptors(new LoggingInterceptor());
//   Registers the LoggingInterceptor for ALL routes in the entire application.
//   Every HTTP request will be logged with method, URL, and response time.

await app.listen(process.env.PORT ?? 3000);
//   Starts the HTTP server on port from .env, or 3000 if not set.
//   ?? is nullish coalescing – uses 3000 only if PORT is null or undefined.
}
bootstrap();
//   Calls the bootstrap function to start the app.
//   It's async because NestFactory.create() and app.listen() return Promises.

```

Teacher may ask:

- **What does `whitelist: true` do?** — Strips properties not defined in the DTO. Prevents mass assignment attacks where someone sends extra malicious fields.
- **What does `forbidNonWhitelisted: true` do?** — Returns 400 error instead of silently stripping. The client knows they sent invalid fields.
- **What does `transform: true` do?** — Converts plain JSON to DTO class instances AND auto-transforms types (e.g., string → number). Required for class-validator to work.
- **Why is `dotenv/config` imported first?** — Because `AppModule` uses `process.env` for database config. If `dotenv` loads after `AppModule`, the env variables would be undefined.
- **What is `useGlobalPipes` vs `useGlobalInterceptors`?** — Pipes transform/validate data BEFORE the handler runs. Interceptors wrap AROUND the handler (before + after).
- **What is the `??` operator?** — Nullish coalescing. It returns the right side only if the left side is `null` or `undefined`. Unlike `||`, it doesn't trigger on `0` or `""`.

app.module.ts — Root Module (Connects Everything)

```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'postgres',
      // Tells TypeORM to use the PostgreSQL database driver

      host: process.env.DB_HOST || 'localhost',
      // Database server hostname. 'localhost' because Podman runs on the same
      machine.

      port: parseInt(process.env.DB_PORT || '5433', 10),
      // Port 5433 because docker-compose.yml maps 5433→5432.
      // parseInt() converts string to number. The '10' means base-10 (decimal).

      username: process.env.DB_USERNAME || 'postgres',
      password: process.env.DB_PASSWORD || 'postgres',
      database: process.env.DB_NAME || 'tp02',
      // These match the POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB
      // defined in docker-compose.yml

      autoLoadEntities: true,
      // Automatically discovers and registers ALL entities that are imported
      // via TypeOrmModule.forFeature() in any sub-module.
      // Without this, you'd need: entities: [Receipt, Order, ...]

      synchronize: true,
      // TypeORM compares your Entity classes with the actual database tables.
      // If there's a mismatch, it AUTO-CREATES or AUTO-ALTERS tables.
      // Example: You add a new @Column() → TypeORM adds the column to the table.
      //
      // WARNING: NEVER use in production!
      // It can DROP columns if you rename/remove them, causing DATA LOSS.
      // In production, use TypeORM migrations instead.
    }),
    ReceiptsModule,
    // Imports the Receipts feature module.
    // This tells NestJS to include all controllers and providers from
    ReceiptsModule.
  ],
  controllers: [AppController],
  // Registers AppController – handles GET / (Hello World route)

  providers: [AppService],
  // Registers AppService – can be injected into AppController
})
export class AppModule {}
```


Teacher may ask:

- **What is `autoLoadEntities`?** — Automatically loads entities registered in `forFeature()`. Without it, you need to list every entity manually in `forRoot({ entities: [...] })`.
- **What is `synchronize: true`?** — Auto-syncs entity definitions with database tables (creates/alters tables). NEVER use in production — use migrations instead.
- **Why port 5433 and not 5432?** — docker-compose maps 5433:5432. Your app connects to `localhost:5433`, which forwards to port 5432 inside the container. This avoids conflicts with any local PostgreSQL.
- **What does `forRoot()` do?** — Sets up the database connection ONE TIME for the entire application. All modules share this connection.
- **What if we have multiple databases?** — You can call `forRoot()` multiple times with different connection names.
- **Why use `process.env` with `|| fallback`?** — Allows configuration via environment variables (for different environments: dev, staging, production) while providing sensible defaults for development.

receipts.entity.ts — Database Entity (The Model)

```
import { Column, Entity, PrimaryGeneratedColumn } from 'typeorm';

@Entity()
// □ Marks this class as a database table.
//   Table name = lowercase class name = "receipt"
//   You can customize: @Entity('my_receipts') → table named "my_receipts"
export class Receipt {

  @PrimaryGeneratedColumn('uuid')
  receiptId: string;
  // □ PRIMARY KEY column that auto-generates a UUID v4.
  //   UUID = Universally Unique Identifier
  //   Example: "60857dee-a7a4-4037-b9ee-78cf6aca96a3"
  //
  //   Why UUID instead of auto-increment integer?
  //   - UUIDs are globally unique (safe for distributed systems)
  //   - Can't guess the next ID (more secure)
  //   - Can generate IDs on client side before saving
  //
  //   Alternative: @PrimaryGeneratedColumn() → auto-increment integer (1, 2, 3...)

  @Column({ type: 'timestamp' })
  issuedAt: Date;
  // □ Stores date AND time. PostgreSQL type: "timestamp without time zone"
  //   Example stored value: "2026-03-19 10:00:00"
  //   TypeORM auto-converts between JavaScript Date and PostgreSQL timestamp.

  @Column()
  name: string;
  // □ Default type is VARCHAR (variable-length string).
  //   PostgreSQL type: "character varying"
  //   NOT NULL by default (TypeORM makes columns NOT NULL unless you say nullable: true)
```

```

@Column({ type: 'decimal', precision: 10, scale: 2 })
price: number;
// □ DECIMAL type for exact money values (no floating point errors).
//   precision: 10 → total number of digits (including decimals)
//   scale: 2 → digits after the decimal point
//   Range: -99999999.99 to 99999999.99
//   PostgreSQL type: "numeric(10,2)"
//
//   Why DECIMAL instead of FLOAT?
//   FLOAT: 0.1 + 0.2 = 0.30000000000000004 (imprecise!)
//   DECIMAL: 0.1 + 0.2 = 0.30 (exact, perfect for money)
}

```

Teacher may ask: - **What is `@PrimaryGeneratedColumn('uuid')`?** — Creates a UUID primary key that is auto-generated by PostgreSQL using `uuid_generate_v4()`. Each receipt gets a unique 36-character ID. - **Why UUID over auto-increment?** — UUIDs are globally unique, harder to guess (security), and work in distributed systems. Auto-increment is simpler but sequential (easy to guess). - **What does `precision: 10, scale: 2` mean?** — 10 total digits, 2 after decimal. Max value: `99999999.99`. This avoids floating-point errors with money. - **What does `@Entity()` do?** — Tells TypeORM "this class = a database table". The table name defaults to the lowercase class name (`receipt`). - **Why is `price a number` in TypeScript but `decimal` in PostgreSQL?** — TypeScript doesn't have a `decimal` type. TypeORM handles the conversion. Note: the value returned from PostgreSQL is actually a string `"4.50"`, but TypeORM can convert it. - **What does `@Column()` without options do?** — Creates a `varchar` (string) column that is **NOT NULL** by default.

receipts.module.ts — Feature Module

```

@Module({
  imports: [TypeOrmModule.forFeature([Receipt])],
  // □ forFeature() registers the Receipt entity's Repository in THIS module's scope.
  //   This creates a Repository<Receipt> provider that can be injected with
  @InjectRepository(Receipt).
  //   Without this line, @InjectRepository(Receipt) would throw an error:
  //   "No repository for Receipt was found. Is it registered in the current scope?"

  controllers: [ReceiptsController],
  // □ Registers ReceiptsController to handle HTTP requests under /receipts

  providers: [ReceiptsService],
  // □ Registers ReceiptsService as a provider (injectable dependency).
  //   NestJS will create ONE instance (singleton) and inject it wherever needed.
})
export class ReceiptsModule {}

```

Teacher may ask: - **What is `forFeature([Receipt])`?** — Registers the TypeORM Repository for the

Receipt entity in this module. Allows `@InjectRepository(Receipt)` in the service. - **Difference between `forRoot()` and `forFeature()`?** - `forRoot()` → called ONCE in AppModule → sets up database CONNECTION - `forFeature()` → called in EACH feature module → registers specific entity repositories - **What is a provider?** — Any class that can be **injected** as a dependency. Services, repositories, guards, interceptors, and factories are all providers. - **What is a singleton?** — NestJS creates only ONE instance of each provider. Every controller/service that injects `ReceiptsService` gets the SAME instance. - **Can we export providers?** — Yes, `exports: [ReceiptsService]` would allow other modules to import ReceiptModule and use its service.

create-receipt.dto.ts — Create Validation

```
import { IsDateString, IsNotEmpty, IsNumber, IsString, Min } from 'class-validator';

export class CreateReceiptDto {
  @IsDateString()
  // □ Validates that the value is a valid ISO 8601 date string.
  //   Valid:   "2026-03-19T10:00:00Z", "2026-03-19", "2026-03-19T10:00:00.000Z"
  //   Invalid: "March 19", "19/03/2026", "not-a-date", 12345
  issuedAt: string;

  @IsString()
  // □ Must be a string type (not number, boolean, object)
  @IsNotEmpty()
  // □ Cannot be an empty string "". Must have at least 1 character.
  //   Without this, "" would pass @IsString() but is not meaningful.
  name: string;

  @IsNumber()
  // □ Must be a number type. Rejects strings like "4.50", booleans, etc.
  //   Note: With transform: true in ValidationPipe, string "4.50" CAN be
  //   converted to number 4.50 automatically.
  @Min(0)
  // □ Must be >= 0. Rejects negative numbers like -5, -0.01.
  //   @Min(0) means minimum value is 0 (inclusive).
  price: number;
}
```

update-receipt.dto.ts — Update Validation

```
import { IsDateString, IsNumber, IsOptional, IsString, Min } from 'class-validator';

export class UpdateReceiptDto {
  @IsOptional()
  // □ This field CAN BE OMITTED from the request body.
  //   If omitted, no validation runs for this field.
}
```

```
// If PROVIDED, it must pass ALL other decorators below it.
@IsDateString()
issuedAt?: string;
// The "?" makes it optional in TypeScript (type: string | undefined)

@IsOptional()
@IsString()
name?: string;

@IsOptional()
@IsNumber()
@Min(0)
price?: number;
}
```

Teacher may ask: - **What is a DTO?** — Data Transfer Object. It's a class that defines the **shape** and **validation rules** for data coming from the client. It acts as a contract between the API and the client. - **Why not validate in the controller or service?** — DTOs separate validation logic from business logic. The ValidationPipe handles validation automatically BEFORE the controller code runs. This is cleaner and follows Single Responsibility Principle. - **Why separate Create and Update DTOs?** — For POST (create), ALL fields are required — you need all data to create a new receipt. For PATCH (update), fields are optional — you might only want to update the name, not the price. - **What happens if validation fails?** — ValidationPipe catches the error and returns a 400 Bad Request with detailed error messages BEFORE the controller method runs. Example: `json { "statusCode": 400, "message": ["price must not be less than 0", "name should not be empty"], "error": "Bad Request" }` - **What is the difference between `@IsNotEmpty()` and `required`?** — A field without `@IsOptional()` is required (must be present). `@IsNotEmpty()` additionally checks it's not an empty string `""`. So: `{ "name": "" }` → passes "required" but fails `@IsNotEmpty()`. - **What is `@IsDateString()`?** — Validates ISO 8601 format. Examples: `"2026-03-19T10:00:00Z"` ✓, `"2026-03-19"` ✓, `"March 19"` ✓, `12345` ✓. - **What does `@Min(0)` do?** — Checks the number is greater than or equal to 0. `@Min(0)` → `0` ✓, `4.50` ✓, `-1` ✗.

receipts.controller.ts — Controller (HTTP Routes)

```
import { Body, Controller, Delete, Get, Param, Patch, Post, UseGuards } from
 '@nestjs/common';
import { ReceiptsService } from './receipts.service';
import { CreateReceiptDto } from './dto/create-receipt.dto';
import { UpdateReceiptDto } from './dto/update-receipt.dto';
import { ApiKeyGuard } from 'src/common/guards/api-key.guard';

@UseGuards(ApiKeyGuard)
// Applies ApiKeyGuard to ALL routes in this controller.
// Every request to /receipts must include a valid x-api-key header.
// If the guard fails → 401 Unauthorized (request never reaches the route handler).
//
// You can also apply guards to individual routes:
```

```

// @UseGuards(ApiKeyGuard)
// @Get()
// findAll() { ... }

@Controller('receipts')
// □ All routes in this controller are prefixed with /receipts.
// @Get() → GET /receipts
// @Get(':id') → GET /receipts/:id
// @Post() → POST /receipts
export class ReceiptsController {

  constructor(private readonly receiptsService: ReceiptsService) {}
  // □ DEPENDENCY INJECTION: NestJS injects ReceiptsService automatically.
  // "private readonly" means:
  // - private: only accessible within this class
  // - readonly: cannot be reassigned after construction
  // This is TypeScript shorthand for declaring + assigning a class property.

  @Get()
  findAll() {
    return this.receiptsService.findAll();
    // □ Calls the service method. NestJS automatically:
    // 1. Awaits the Promise (since service methods are async)
    // 2. Serializes the result to JSON
    // 3. Sets status code 200
    // 4. Sends the response
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    // □ @Param('id') extracts the ':id' parameter from the URL.
    // GET /receipts/abc-123 → id = "abc-123"
    // The parameter name in @Param('id') must match the ':id' in @Get(':id')
    return this.receiptsService.findOne(id);
  }

  @Post()
  create(@Body() dto: CreateReceiptDto) {
    // □ @Body() extracts the entire JSON request body.
    // The body is validated against CreateReceiptDto by ValidationPipe.
    // If validation fails → 400 error (this method never runs).
    // If validation passes → dto is a CreateReceiptDto instance with validated
data.
    //
    // NestJS auto-sets status 201 for @Post() (Created).
    return this.receiptsService.create(dto);
  }

  @Patch(':id')
  update(@Param('id') id: string, @Body() dto: UpdateReceiptDto) {
    // □ Combines URL parameter AND body extraction.

```

```

    // PATCH /receipts/abc-123 with body { "name": "Espresso" }
    // → id = "abc-123", dto = { name: "Espresso" }
    return this.receiptsService.update(id, dto);
  }

  @Delete('/:id')
  remove(@Param('id') id: string) {
    return this.receiptsService.remove(id);
  }
}

```

Teacher may ask: - **Why PATCH and not PUT?** - **PUT** = replace the ENTIRE resource. Client must send ALL fields. Missing fields are set to null. - **PATCH** = PARTIAL update. Client sends only the fields they want to change. Other fields remain unchanged. - Since **UpdateReceiptDto** has all optional fields, PATCH is the correct method. - **What does @Param('id') do?** — Extracts the named parameter from the URL path. **GET /receipts/abc-123** → **id = "abc-123"**. - **What does @Body() do?** — Extracts the JSON request body and converts it to the DTO class instance (because **transform: true** in **ValidationPipe**). - **What does @UseGuards() at class level do?** — Applies the guard to ALL routes. You can also use it at method level for specific routes only. - **Why does @Post() return 201 and @Get() returns 200?** — NestJS uses appropriate defaults: POST → 201 Created, everything else → 200 OK. You can override with **@HttpCode(204)**. - **What is private readonly?** — TypeScript constructor shorthand. **private** = only this class can access it. **readonly** = cannot be reassigned. It automatically creates a class property and assigns the parameter to it. - **Can a controller call another controller?** — No! Controllers should call services. If two controllers need the same logic, they should share a service. - **What if the controller returns a Promise?** — NestJS automatically awaits it and sends the resolved value as JSON.

receipts.service.ts — Service (Business Logic Layer)

```

import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Receipt } from 'src/database/entities/receipts.entity';
import { CreateReceiptDto } from '../dto/create-receipt.dto';
import { UpdateReceiptDto } from '../dto/update-receipt.dto';

@Injectable()
// □ Marks this class as a NestJS provider that can be injected.
// Without @Injectable(), NestJS cannot manage this class in the DI container.
export class ReceiptsService {

  constructor(
    @InjectRepository(Receipt)
    private readonly receiptRepo: Repository<Receipt>,
    // □ Injects the TypeORM Repository for the Receipt entity.
    // Repository<Receipt> provides methods: find, findOne, create, save, remove,

```

```

etc.
    // @InjectRepository(Receipt) tells NestJS WHICH entity's repository to inject.
  ) {}

  async findAll() {
    return this.receiptRepo.find({ order: { issuedAt: 'DESC' } });
    //   SQL: SELECT * FROM receipt ORDER BY "issuedAt" DESC
    //   Returns ALL receipts sorted by issuedAt date, newest first.
    //   find() returns an array. If no receipts exist, returns [].
  }

  async findOne(receiptId: string) {
    const receipt = await this.receiptRepo.findOne({ where: { receiptId } });
    //   SQL: SELECT * FROM receipt WHERE "receiptId" = $1 LIMIT 1
    //   { where: { receiptId } } is shorthand for { where: { receiptId: receiptId } }
    //   Returns the receipt object, or NULL if not found.

    if (!receipt) throw new NotFoundException('Receipt not found');
    //   If no receipt found, throw NotFoundException.
    //   NestJS catches this and returns:
    //   { "statusCode": 404, "message": "Receipt not found", "error": "Not Found" }
    //
    //   This method is also used by update() and remove() to check existence.
    //   This avoids duplicating the "check if exists" logic.

    return receipt;
  }

  async create(dto: CreateReceiptDto) {
    const receipt = this.receiptRepo.create({
      issuedAt: new Date(dto.issuedAt),
      //   Converts ISO string "2026-03-19T10:00:00Z" to JavaScript Date object.
      //   The entity expects Date type, but DTO receives string from client.
      name: dto.name,
      price: dto.price,
    });
    //   create() makes a Receipt instance IN MEMORY only.
    //   No SQL is executed. No database interaction yet.
    //   The receiptId is NOT generated yet (it's undefined).

    return this.receiptRepo.save(receipt);
    //   save() executes the actual SQL INSERT query.
    //   SQL: INSERT INTO receipt ("issuedAt", "name", "price") VALUES ($1, $2, $3)
    RETURNING *
    //   PostgreSQL generates the UUID for receiptId.
    //   Returns the full receipt object WITH the generated receiptId.
  }

  async update(receiptId: string, dto: UpdateReceiptDto) {
    const receipt = await this.findOne(receiptId);
    //   First, find the existing receipt (throws 404 if not found).

```

```

// This reuses the findOne() method above.

if (dto.issuedAt !== undefined) receipt.issuedAt = new Date(dto.issuedAt);
if (dto.name !== undefined) receipt.name = dto.name;
if (dto.price !== undefined) receipt.price = dto.price;
// Only update fields that were actually provided in the request body.
// If client sends { "name": "Espresso" }, only name changes.
// price and issuedAt remain unchanged.
// We check "!== undefined" because the field might not be in the body at all.
//
// Why not just Object.assign(receipt, dto)?
// Because dto.issuedAt is a string but receipt.issuedAt needs to be a Date.
// We need the new Date() conversion.

return this.receiptRepo.save(receipt);
// save() on an EXISTING entity (one with a receiptId) executes UPDATE.
// SQL: UPDATE receipt SET "name" = $1 WHERE "receiptId" = $2 RETURNING *
// TypeORM knows to UPDATE (not INSERT) because receiptId already exists.
}

async remove(receiptId: string) {
  const receipt = await this.findOne(receiptId);
  // Verify the receipt exists (throws 404 if not)

  await this.receiptRepo.remove(receipt);
  // SQL: DELETE FROM receipt WHERE "receiptId" = $1
  // remove() expects an entity instance, not just an ID.
  // After removal, the receipt object still exists in memory but is deleted from
  DB.

  return { deleted: true, receiptId };
  // Returns a confirmation object. The receipt is already gone from the database.
  // We return the receiptId so the client knows which receipt was deleted.
}
}

```

Teacher may ask: - **What is `@Injectable()`?** — A decorator that marks the class as a provider in the NestJS DI system. Without it, NestJS cannot inject this class. - **Difference between `create()` and `save()`?** - `create()` = makes an object in JavaScript memory. NO database query. - `save()` = writes to database. If entity has no ID → INSERT. If entity has an ID → UPDATE. - **How does `save()` know to INSERT vs UPDATE?** — It checks if the primary key (`receiptId`) exists. If it does → UPDATE. If it doesn't → INSERT. - **What is `NotFoundException`?** — A built-in NestJS exception that returns HTTP 404. NestJS has exceptions for every HTTP status: `BadRequestException` (400), `UnauthorizedException` (401), `ForbiddenException` (403), `NotFoundException` (404), `InternalServerErrorException` (500). - **Why does `update()` call `findOne()` first?** — Two reasons: (1) Verify the receipt exists (throw 404 if not). (2) Get the current entity so we can modify only the changed fields. - **Why check `dto.issuedAt !== undefined`?** — Because `@IsOptional()` means the field might not be present. If client sends { "name": "Tea" }, then `dto.price` is `undefined`. We don't want to overwrite price with `undefined`. - **What does `async` do?** — Makes the function return a Promise. All database operations are asynchronous (they

take time), so we use `async/await` to wait for them. - **What does `remove()` vs `delete()` do?** — `remove()` takes an entity instance and deletes it. `delete()` takes an ID/criteria and deletes without loading the entity first. We use `remove()` because we want to verify existence first.

api-key.guard.ts — Authentication Guard

```
import { CanActivate, ExecutionContext, Injectable, UnauthorizedException } from
 '@nestjs/common';

@Injectable()
export class ApiKeyGuard implements CanActivate {
  // □ implements CanActivate → must have a canActivate() method.
  //   CanActivate is an interface that tells NestJS "this is a guard".

  canActivate(context: ExecutionContext): boolean {
    // □ This method runs BEFORE the route handler.
    //   Return true → request proceeds to the handler.
    //   Throw exception → request is rejected immediately.

    const req = context.switchToHttp().getRequest<Request & { headers: any }>();
    // □ ExecutionContext is a wrapper around the request.
    //   switchToHttp() tells NestJS "this is an HTTP context" (not
    WebSocket/GraphQL).
    //   getRequest() returns the Express Request object.

    const apiKey = req.headers['x-api-key'];
    // □ Reads the "x-api-key" custom header from the HTTP request.
    //   Headers are case-insensitive in HTTP (X-API-Key = x-api-key).
    //   Custom headers typically use "x-" prefix.

    if (!apiKey || apiKey !== process.env.API_KEY) {
      throw new UnauthorizedException('Invalid API key');
      // □ Throws 401 Unauthorized if:
      //   1. No x-api-key header was sent (!apiKey)
      //   2. The key doesn't match the one in .env (apiKey !== process.env.API_KEY)
      //
      //   Response: { "statusCode": 401, "message": "Invalid API key", "error":
      "Unauthorized" }
    }
    return true;
    // □ API key is valid → allow the request to proceed to the controller.
  }
}
```

Teacher may ask: - What is a Guard? — A class that implements `CanActivate` interface. It decides whether a request should be handled (return `true`) or rejected (throw exception). Guards run AFTER middleware but BEFORE interceptors and pipes. - **What is `CanActivate`?** — A NestJS interface

requiring a `canActivate()` method. It returns `boolean` or `Promise<boolean>`. If `true`, the request continues. If `false` or throws, the request is denied. - **What is `ExecutionContext`?** — An object that wraps the incoming request and provides information about the current execution: the controller class, the handler method, the request type (HTTP/WS/RPC), and the request/response objects. - **What does `switchToHttp()` do?** — NestJS supports HTTP, WebSockets, and microservices. `switchToHttp()` tells NestJS "I want to work with HTTP request/response objects specifically." - **Why not use middleware for authentication?** — Guards have access to `ExecutionContext` which knows which controller and method will handle the request. This allows for role-based access control. Middleware only has access to `req`, `res`, `next`. - **What is `process.env.API_KEY`?** — An environment variable loaded from the `.env` file. The actual API key value is stored outside the code for security. Never hardcode secrets in source code. - **Difference between 401 and 403?** - **401 Unauthorized** = "Who are you? You didn't authenticate." (no API key or wrong key) - **403 Forbidden** = "I know who you are, but you don't have permission." (valid key but no access) - **Where is the guard applied?** — On `ReceiptsController` using `@UseGuards(ApiKeyGuard)`. `AppController` does NOT have the guard, so `GET /` works without an API key.

logging.interceptor.ts — Logging Interceptor

```
import {
  CallHandler,
  ExecutionContext,
  Injectable,
  NestInterceptor,
} from '@nestjs/common';
import { Observable } from 'rxjs';
import { tap } from 'rxjs/operators';

@Injectable()
export class LoggingInterceptor implements NestInterceptor {
  // □ implements NestInterceptor → must have an intercept() method.

  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    // □ This method wraps AROUND the route handler.
    //   Code BEFORE next.handle() runs BEFORE the handler.
    //   Code INSIDE .pipe() runs AFTER the handler.

    const req = context.switchToHttp().getRequest();
    const { method, url } = req;
    // □ Destructures the HTTP method (GET, POST, etc.) and URL (/receipts, etc.)
    //   from the Express request object.

    const start = Date.now();
    // □ Records the timestamp BEFORE the handler runs (in milliseconds).

    return next.handle().pipe(
      // □ next.handle() calls the actual route handler (e.g., findAll()).
      //   It returns an Observable (RxJS stream).
```

```
// .pipe() lets you add operators to run after the handler completes.

tap(() => {
  const ms = Date.now() - start;
  // □ Calculates how long the handler took to execute.
  console.log(`[HTTP] ${method} ${url} - ${ms}ms`);
  // □ Logs to console. Example output:
  //   [HTTP] GET /receipts - 12ms
  //   [HTTP] POST /receipts - 45ms
  //   [HTTP] DELETE /receipts/abc-123 - 8ms
}),
);
// □ tap() is a "side effect" operator – it does something (logging)
//   WITHOUT modifying the response data. The client gets the same response
//   as if the interceptor wasn't there.
}
}
```

Teacher may ask: - What is an Interceptor? — A class that implements `NestInterceptor`. It wraps AROUND the route handler, allowing you to run logic BEFORE and AFTER the handler. Use cases: logging, caching, response transformation, error handling. - **What is `next.handle()`?** — It invokes the route handler (controller method). Returns an Observable that emits the handler's return value. - **What is an Observable?** — An RxJS concept representing a stream of values over time. In NestJS, each request is treated as an Observable that emits one value (the response). - **What is `tap()`?** — An RxJS operator that executes a side effect (like logging) without changing the emitted value. The response data passes through unchanged. - **What is `pipe()`?** — Chains RxJS operators. `observable.pipe(tap(...), map(...))` applies operators in sequence. - **Guard vs Interceptor vs Pipe** — **What's the difference?** | Feature | Guard | Interceptor | Pipe | |——|——|———|——| | **Purpose** | Allow/deny request | Add logic before/after handler | Transform/validate data | | **Runs** | Before everything | Before + after handler | Before handler (on params/body) | | **Can modify response?** | No | Yes (using `map()`) | No (only input) | | **Example** | API key check | Logging, caching | Validation, type conversion | - **What is `Date.now()`?** — Returns current timestamp in milliseconds. By recording it before and after the handler, we calculate response time. - **Could we use middleware instead of an interceptor for logging?** — Yes, but interceptors have access to the Observable response stream, so they can also log the response body, status code, or transform responses. Middleware only has `req`, `res`, `next`.

docker-compose.yml — Database Container Configuration

```
version: '3.8'
# □ Docker Compose file format version.

services:
  tp02-postgres:
    # □ Service name (internal reference). Used for docker networking.
```

```

image: postgres:16
# □ Uses the official PostgreSQL 16 image from Docker Hub.
#   The image includes the PostgreSQL server, ready to run.

container_name: tp02_postgres
# □ Gives the container a fixed name (instead of auto-generated).
#   Used to reference the container: podman exec tp02_postgres psql ...

environment:
  POSTGRES_USER: postgres
  # □ Creates a superuser with this username.

  POSTGRES_PASSWORD: postgres
  # □ Sets the password for the superuser.

  POSTGRES_DB: tp02
  # □ Automatically creates a database named "tp02" on first start.
  #   Without this, only the default "postgres" database exists.

ports:
  - '5433:5432'
  # □ PORT MAPPING: hostPort:containerPort
  #   Container runs PostgreSQL on 5432 (default).
  #   Host machine accesses it on 5433.
  #   Your NestJS app connects to localhost:5433.
  #   Traffic flow: App → localhost:5433 → container:5432 → PostgreSQL

volumes:
  - tp02_pgdata:/var/lib/postgresql/data
  # □ NAMED VOLUME: Persists database files outside the container.
  #   /var/lib/postgresql/data = where PostgreSQL stores its data files.
  #   tp02_pgdata = a named volume managed by Docker/Podman.
  #
  #   Without this volume:
  #   - Stop container → data is preserved (in container)
  #   - Remove container → ALL DATA IS LOST
  #
  #   With this volume:
  #   - Remove container → data is safe in the volume
  #   - Create new container with same volume → data restored

volumes:
  tp02_pgdata:
  # □ Declares the named volume. Podman/Docker manages its storage location.

```

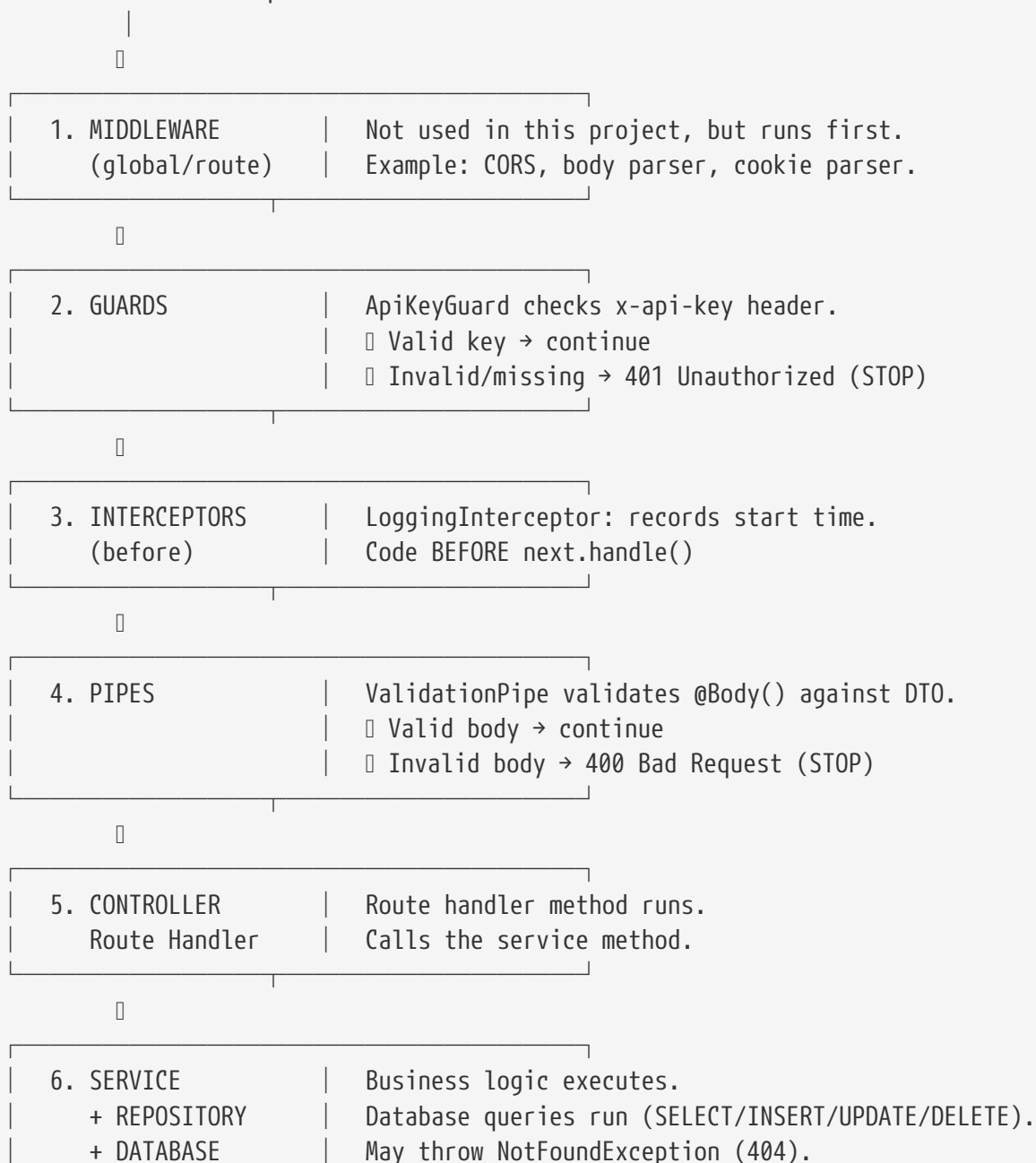
Teacher may ask: - What is Docker Compose / Podman Compose? — A tool to define and run multi-container applications. You describe services in YAML, then start everything with **docker compose up** or **podman compose up**. - **Why use containers?** — Consistency (same environment everywhere), isolation (doesn't affect your host system), easy setup (one command), easy cleanup

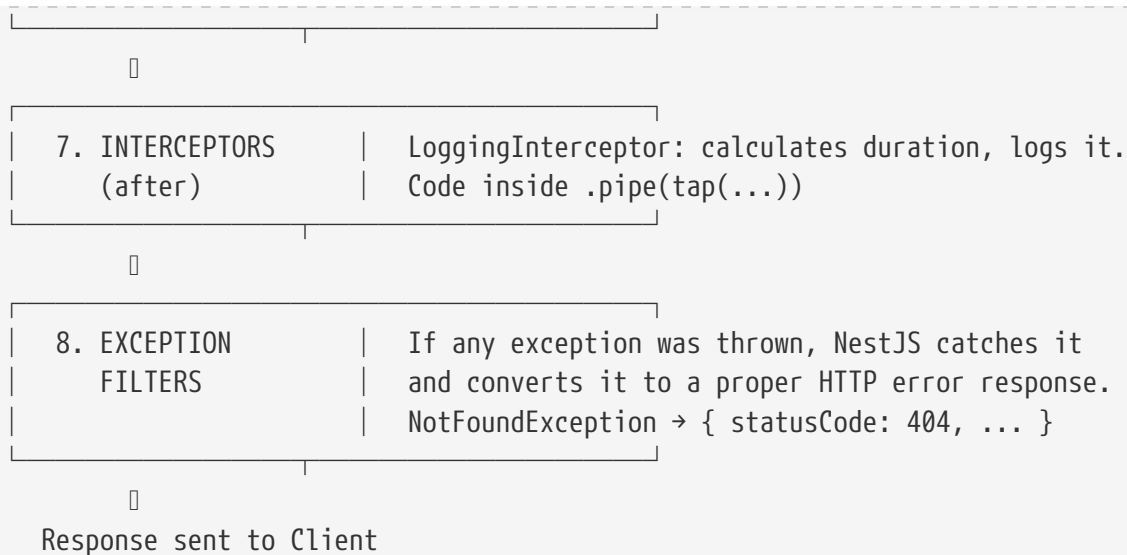
(remove container = clean uninstall). - **What does the volume do?** — Stores database files on the host machine. Without volumes, deleting the container deletes ALL data. With volumes, data survives container deletion. - **What does 5433:5432 mean?** — Maps host port 5433 to container port 5432. The app connects to `localhost:5433`, which forwards to PostgreSQL inside the container on port 5432. - **Why not use port 5432 directly?** — To avoid conflicts with any local PostgreSQL installation that might already use port 5432. - **What happens when you run `podman compose up -d`** — `-d` = detached mode (runs in background). It pulls the PostgreSQL image (if not cached), creates the container, creates the volume, and starts PostgreSQL.

□ NestJS Request Lifecycle (Detailed)

Understanding the order of execution is CRITICAL for teacher questions:

Client sends HTTP Request





Key point: If a Guard fails (step 2), steps 3-7 NEVER run. If a Pipe fails (step 4), steps 5-7 NEVER run. Each step acts as a gate.

□ API Test Cases (TP02.http) Detailed Explanation

Test 1: GET / — Hello World

```
GET http://localhost:3000/
```

- **Expected:** 200 OK, body: "Hello World!"
- **Why it works:** `AppController.getHello()` → `AppService.getHello()` → returns "Hello World!"
- **No API key needed** because `@UseGuards(ApiKeyGuard)` is only on `ReceiptsController`, not `AppController`

Test 2: GET /receipts — Without API Key

```
GET http://localhost:3000/receipts
```

- **Expected:** 401 Unauthorized
- **Why:** `ApiKeyGuard` runs first, finds no `x-api-key` header → throws `UnauthorizedException`
- **Response body:** { "statusCode": 401, "message": "Invalid API key", "error": "Unauthorized" }

Test 3: GET /receipts — With API Key

```
GET http://localhost:3000/receipts
x-api-key: my-secret-api-key-123
```

- **Expected:** 200 OK, body: array of receipts [...]
- **Why:** Guard passes → `findAll()` → `SELECT * FROM receipt ORDER BY "issuedAt" DESC`
- **If no receipts exist:** Returns empty array []

Test 4: POST /receipts — Create a Receipt

```
POST http://localhost:3000/receipts
Content-Type: application/json
x-api-key: my-secret-api-key-123

{ "issuedAt": "2026-03-19T10:00:00Z", "name": "Coffee", "price": 4.50 }
```

- **Expected:** 201 Created, body: the created receipt with generated UUID
- **Flow:** Guard □ → `ValidationPipe` validates all 3 fields □ → `create()` → INSERT into database → returns receipt with UUID
- **Response body example:** ``json { "receiptId": "60857dee-a7a4-4037-b9ee-78cf6aca96a3", "issuedAt": "2026-03-19T10:00:00.000Z", "name": "Coffee", "price": "4.50" }``

Test 5: GET /receipts/:id — Get One Receipt

```
GET http://localhost:3000/receipts/60857dee-a7a4-4037-b9ee-78cf6aca96a3
x-api-key: my-secret-api-key-123
```

- **Expected:** 200 OK, body: single receipt object
- **Why:** `findOne()` → `SELECT * FROM receipt WHERE "receiptId" = $1`
- **Important:** Replace the UUID with one from Test 4's response

Test 6: PATCH /receipts/:id — Update a Receipt

```
PATCH http://localhost:3000/receipts/60857dee-a7a4-4037-b9ee-78cf6aca96a3
Content-Type: application/json
x-api-key: my-secret-api-key-123

{ "name": "Espresso", "price": 5.00 }
```

- **Expected:** 200 OK, body: updated receipt
- **Why:** `findOne()` finds existing receipt → updates only `name` and `price` → `issuedAt` remains unchanged → saves
- **This is a PARTIAL update** — we only sent 2 of 3 fields, and `issuedAt` was not affected

Test 7: DELETE /receipts/:id — Delete a Receipt

```
DELETE http://localhost:3000/receipts/60857dee-a7a4-4037-b9ee-78cf6aca96a3
x-api-key: my-secret-api-key-123
```

- **Expected:** 200 OK, body: `{ "deleted": true, "receiptId": "60857dee-..." }`
- **Why:** `findOne()` verifies receipt exists → `remove()` deletes from database → returns confirmation

Test 8: GET /receipts/:id — Not Found

```
GET http://localhost:3000/receipts/00000000-0000-0000-0000-000000000000
x-api-key: my-secret-api-key-123
```

- **Expected:** 404 Not Found
- **Why:** `findOne()` returns null → `throw new NotFoundException('Receipt not found')`
- **Response body:** `{ "statusCode": 404, "message": "Receipt not found", "error": "Not Found" }`

Test 9: POST /receipts — Invalid Body

```
POST http://localhost:3000/receipts
Content-Type: application/json
x-api-key: my-secret-api-key-123

{ "name": "", "price": -5 }
```

- **Expected:** 400 Bad Request
- **Why — 3 validation errors:**
 1. `issuedAt` is **missing** (required field, no `@IsOptional()`)
 2. `name` is `""` → fails `@IsNotEmpty()`
 3. `price` is `-5` → fails `@Min(0)`
- **Response body example:** ``json { "statusCode": 400, "message": [ "issuedAt must be a valid ISO 8601 date string", "name should not be empty", "price must not be less than 0" ], "error": "Bad Request" } ``

□ Summary of Key Decorators

NestJS Core Decorators

Decorator	Where	Purpose
<code>@Module({ ... })</code>	Class	Defines a module — the basic organizational unit in NestJS
<code>@Controller('path')</code>	Class	Defines a controller with a base route prefix
<code>@Injectable()</code>	Class	Marks class as a provider that can be injected via DI
<code>@Get()</code> , <code>@Post()</code> , <code>@Patch()</code> , <code>@Delete()</code>	Method	Maps a method to an HTTP method + route
<code>@Body()</code>	Parameter	Extracts the request body
<code>@Param('name')</code>	Parameter	Extracts a URL path parameter
<code>@UseGuards(GuardClass)</code>	Class/Method	Applies a guard for authentication/authorization

TypeORM Decorators

Decorator	Where	Purpose
<code>@Entity()</code>	Class	Maps class to a database table
<code>@PrimaryGeneratedColumn('uuid')</code>	Property	Auto-generated UUID primary key
<code>@Column({ type, precision, scale })</code>	Property	Maps property to a table column
<code>@InjectRepository(Entity)</code>	Constructor param	Injects TypeORM repository for an entity

class-validator Decorators

Decorator	Purpose	Example Failure
<code>@IsString()</code>	Must be string type	<code>123</code> → □
<code>@IsNumber()</code>	Must be number type	<code>"abc"</code> → □
<code>@IsDateString()</code>	Must be ISO 8601 date	<code>"March 19"</code> → □
<code>@IsNotEmpty()</code>	String cannot be ""	<code>""</code> → □
<code>@IsOptional()</code>	Field can be omitted	<code>undefined</code> → □ (skip validation)
<code>@Min(0)</code>	Number must be ≥ 0	<code>-5</code> → □

□ Common Teacher Questions & Answers

General Architecture

1. **Q: What is the purpose of NestJS modules?** A: Modules organize the application into cohesive blocks. Each module encapsulates related controllers, services, and entities. This promotes separation of concerns and makes the app scalable.
2. **Q: What is the difference between controllers and providers?** A: Controllers handle HTTP requests and define routes. Providers (services) contain business logic and can be injected into controllers or other providers.
3. **Q: Why do we separate controller and service?** A: Single Responsibility Principle. Controller handles HTTP (params, body, headers). Service handles logic (validation, database queries). Services can be reused across multiple controllers.

Dependency Injection

1. **Q: How does DI work without `new` keyword?** A: NestJS IoC container reads the constructor parameters, finds matching providers registered in the module, creates instances, and injects them automatically.
2. **Q: What happens if you forget `@Injectable()`?** A: NestJS throws an error at startup because it cannot resolve the dependency. The DI container doesn't know about the class.

Database

1. **Q: Why not write raw SQL?** A: TypeORM provides type safety, prevents SQL injection, handles connection pooling, and makes code database-agnostic (can switch from PostgreSQL to MySQL easily).
2. **Q: What is the danger of `synchronize: true` in production?** A: It can drop columns or tables when entity definitions change, causing permanent data loss. In production, use migrations — controlled, versioned database changes.

Security

1. **Q: Where is the API key stored?** A: In a `.env` file as `API_KEY=my-secret-api-key-123`. The `.env` file should be in `.gitignore` so it's never committed to version control.
2. **Q: Is API key authentication secure enough for production?** A: For simple internal APIs, yes. For public APIs, consider JWT tokens, OAuth2, or session-based authentication for better security.